Многопоточность и корутины

Потоки, колбэки и suspend-функции



Разработка мобильных приложений на Android

Почему корутины устроены именно так

Что должно стать понятным

- Чем отличаются конкурентность, параллельность и асинхронность
- Что такое процесс и поток и почему потоки дорогие
- Почему «поток на запрос» и колбэки упёрлись в пределы
- Что такое корутина и что она решила

Результат: объяснить, что происходит с потоком, когда корутина «ждёт».

Нажали «Загрузить»

Запрос к серверу идёт 2 секунды.
Что всё это время делает экран?

А если пользователь в это время листает список? А если таких запросов десять?

Слова, которые путают

- Конкурентность — concurrency
- Параллельность — parallelism
- Асинхронность — asynchrony
- Многопоточность — multithreading
- Блокирующий и неблокирующий вызов

Это не синонимы: каждое слово отвечает на свой вопрос.

Конкурентность и параллельность

Конкурентность: одно ядро, задачи чередуются



обе задачи «в работе», но в каждый момент выполняется одна

Параллельность: два ядра, задачи одновременно



задачи выполняются в один и тот же момент

Конкурентность — про устройство программы. Параллельность — про исполнение.

Иметь дело и делать

Конкурентность

Несколько задач в работе одновременно.

Могут чередоваться на одном ядре.

Свойство программы.

Параллельность

Несколько задач выполняются в один момент.

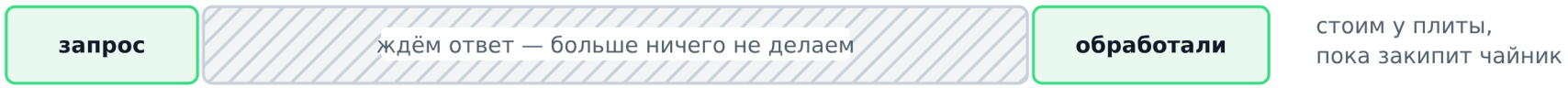
Нужно несколько ядер.

Свойство исполнения.

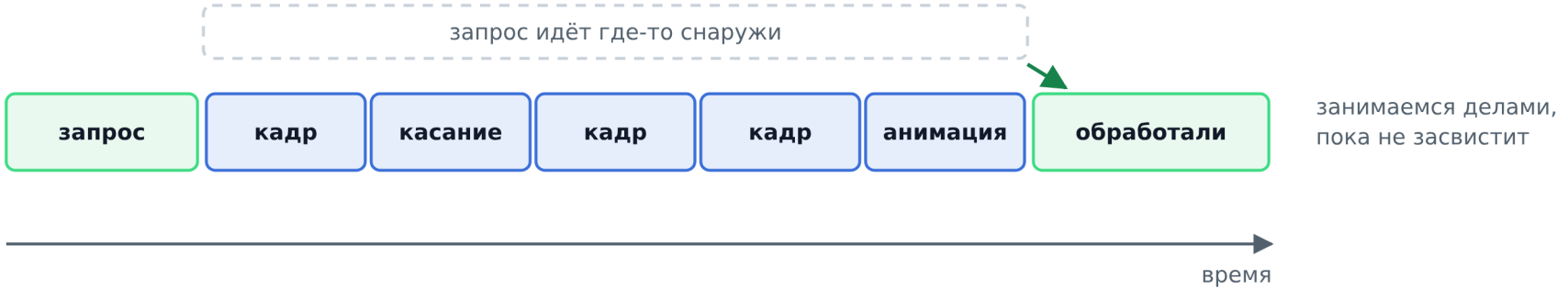
Конкурентность — иметь дело со многими задачами. Параллельность — делать многое сразу.

Синхронно и асинхронно

Синхронно: ждём на месте



Асинхронно: не ждём, результат придёт позже



Асинхронность — не ждать результат на месте, а получить его позже.

Пять слов — пять вопросов

Термин	Вопрос	Пример
Конкурентность	Сколько задач в работе?	UI отвечает, пока идёт загрузка
Параллельность	Выполняются в один момент?	Сжатие фото на 8 ядрах
Асинхронность	Ждём результат на месте?	Ответ придёт позже
Блокирующий вызов	Поток занят, пока ждём?	Thread.sleep, чтение файла
Многопоточность	Сколько потоков исполнения?	Главный поток и фоновые

Асинхронный код может работать в одном потоке, а многопоточный — быть синхронным.

Конкурентность без многопоточности?

Один поток.
Сотни задач по очереди.

Так работает главный поток Android и весь JavaScript в браузере. Как это возможно?

Что такое поток?

Мы говорим «запустить в другом потоке».
А что такое поток?

И чем он отличается от процесса?

Процесс

- Запущенная программа
- Своя память: другие процессы её не видят
- Свои ресурсы: открытые файлы, сетевые соединения
- На Android у каждого приложения свой процесс Linux

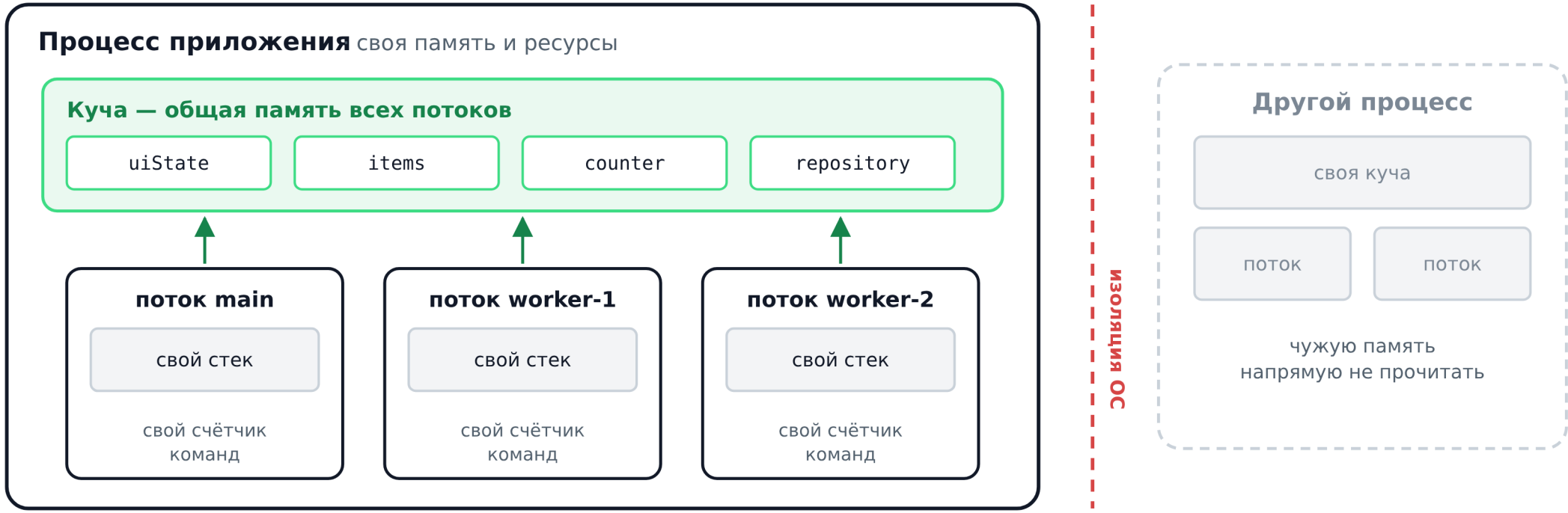
Процесс — контейнер: память и ресурсы.

Поток

- Последовательность инструкций, которая выполняется внутри процесса
- У каждого потока свой стек и счётчик команд
- Потоки одного процесса делят общую память — кучу
- Какой поток и на каком ядре выполнять, решает планировщик ОС

Процесс — то, внутри чего. Поток — то, что выполняется.

Процесс и потоки в памяти



Объекты в куче видят все потоки процесса. Стек у каждого потока свой.

Процесс и поток

	Процесс	Поток
Память	Своя, изолирована	Общая с потоками процесса
Обмен данными	Через механизмы ОС	Через общие объекты
Создание	Дорого	Дешевле, но не бесплатно
Ошибка	Не задевает другие процессы	На Android падает всё приложение

Общая память — и сила потоков, и источник ошибок.

Сколько стоит поток

- Стек — на JVM по умолчанию порядка 1 МБ
- Создание и завершение — обращения к ОС
- Переключение между потоками — сохранить и восстановить состояние
- Тысячи потоков — нормально, сотни тысяч — уже нет

Поток — дорогой ресурс ОС. Много их не создашь.

Поток в Kotlin

```
import kotlin.concurrent.thread

fun main() {
    val worker = thread {
        Thread.sleep(1000)
        println("Готово")
    }
    println("main не ждёт")
    worker.join()
}
```

thread { } создаёт и сразу запускает поток.

Thread.sleep блокирует только worker.

join — дождаться завершения.

Сначала выведется «main не ждёт», через секунду — «Готово».

Два потока, один счётчик

Каждый поток 100 000 раз делает counter++.

Сколько получится в итоге?

Гонка

```
var counter = 0

fun main() {
    val a = thread { repeat(100_000) { counter++ } }
    val b = thread { repeat(100_000) { counter++ } }
    a.join()
    b.join()
    println(counter)
}
```

Ожидаем 200 000.

Получаем 127 384, 151 902, 143 117…

Каждый запуск — разный результат.

Гонка (race condition): результат зависит от того, как переключались потоки.

Почему теряются увеличения



Два увеличения — а счётчик вырос на 1. Одно обновление потеряно.

counter++ — не одна операция, а три. Между ними может вклиниться другой поток.

Как починить

```
// Атомарная операция
val atomic = AtomicInteger(0)
thread {
    repeat(100_000) { atomic.incrementAndGet() }
}

// Блокировка: в секции один поток
val lock = Any()
var counter = 0
thread {
    repeat(100_000) {
        synchronized(lock) { counter++ }
    }
}
```

Теперь всегда 200 000.

Но блокировка — это ожидание.

А две блокировки — риск deadlock.

Синхронизация чинит гонку, но делает код медленнее и сложнее.

Как жить с общими данными

Подход	Идея	Цена
Блокировки	В секцию входит один поток	Ожидание, риск deadlock
Атомарные типы	Операция неделима	Только простые операции
Неизменяемые данные	Нечего портить	Изменение — новая копия
Один владелец	Данные меняет один поток	Нужно передавать ему работу

Лучше всего — вообще не делить изменяемые данные между потоками.

На каком потоке наш onClick?

А composable-функции?
А onCreate?

И кто в это время рисует кадры и обрабатывает касания?

Главный поток Android

- Один на приложение, создаётся при запуске процесса
- Обрабатывает касания, отрисовку, колбэки Activity
- В нём выполняются composable-функции и наши onClick
- Работает по очереди: берёт следующее сообщение из очереди
- Кадр — примерно каждые 16 мс при 60 Гц

Главный поток нельзя занимать надолго.

Главный поток — очередь сообщений



Пока выполняется наш код, кадры не рисуются и касания ждут.

Если занять главный поток

Сколько занят	Что увидит пользователь
Дольше кадра (16 мс)	Пропущенный кадр, рывок анимации
Сотни миллисекунд	Интерфейс заметно замирает
5 секунд без ответа на касание	ANR: «Приложение не отвечает»
Сетевой запрос	NetworkOnMainThreadException сразу

Главный поток — только для UI. Ожидание и тяжёлые вычисления — в другом месте.

Задача, которую нужно решить

```
fun onLoadClick() {                // главный поток
    val items = api.loadItems()    // сеть: 2 с
    uiState = uiState.copy(items = items)
}
```

Сеть на главном потоке — исключение.

Если бы не оно — замерший экран и ANR.

Нужно: уйти с главного потока, а с результатом — вернуться на него.

Решение «в лоб»: поток и Handler

```
fun onLoadClick() {
    thread {                    // фон
        val items = api.loadItems()
        mainHandler.post {      // главный
            uiState = uiState.copy(items = items)
        }
    }
}
```

Работает. Но:

как отменить, если экран закрыли?

куда деть ошибку?

сколько потоков даст десять нажатий?

Каждый из этих вопросов приходится решать вручную.

Как это делали в Android

Инструмент	Что было не так
Thread и Handler	Отмена, ошибки, повороты — вручную
AsyncTask	Утечки Activity, потеря результата; устарел в API 30
Loaders	Громоздкая связка с Activity и Fragment
RxJava	Мощно, но высокий порог входа
Корутины	Рекомендованный способ сейчас

Android годами искал удобный способ «сходить в фон и вернуться».

Сервер и 10 000 пользователей

10 000 пользователей одновременно ждут ответа.

Сколько потоков нужно серверу? Проблема та же, что на телефоне, только больше.

Поток на запрос

- Каждый входящий запрос получает свой поток
- Код простой: прочитал запрос, сходил в БД, ответил
- Пока ждём БД или другой сервис, поток просто спит
- Так долгие годы работали Java-серверы: сервлеты, Tomcat

Простой код — за счёт потока, который почти всё время ждёт.

Обработчик запроса

```
fun handle(request: Request): Response {
    val user = db.findUser(request.userId) // 20 мс
    val orders = ordersApi.load(user.id)   // 100 мс
    val total = orders.sumOf { it.price }  // 0,01 мс
    return Response(user.name, total)
}
```

Читается сверху вниз.

Из 120 мс поток работает 0,01 мс.

Остальное время — занят ожиданием.

Поток — дорогой ресурс, а тратится на ожидание.

Два способа обслужить много запросов

Поток на запрос



Потоков столько, сколько запросов.
Почти всё время они ждут.

Неблокирующий ввод-вывод



Начали запрос 1 → не ждём, начали 2, 3, 4.
Пришёл ответ 2 → вызвали его колбэк.
Пришёл ответ 1 → вызвали его колбэк…

Потоков мало, запросов тысячи.
Но вместо «сделай, потом сделай» —
«сделай и позови меня, когда будет готово».

Так устроены Node.js, Netty, nginx

Либо много потоков, которые ждут, либо мало потоков и код на колбэках.

Где предел

- 10 000 соединений — 10 000 потоков
- Память на стеки и время на переключения
- Пул ограничен — остальные запросы ждут в очереди
- Задача C10K: 10 000 соединений на одном сервере (1999)

Узкое место — не процессор, а количество ждущих потоков.

Выход: не ждать, а подписаться

- Неблокирующий ввод-вывод: «сообщи, когда ответ придёт»
- Несколько потоков обслуживают тысячи соединений
- Результат приходит в функцию-обработчик — колбэк
- Код больше не читается сверху вниз

Потоков стало мало, а код стал сложным.

Колбэк

```
fun loadUser(id: Int, onResult: (User) -> Unit)

loadUser(42) { user ->
    showUser(user)
}
println("Запрос отправлен")  // раньше showUser
```

Функция возвращается сразу.

Результат — когда-нибудь потом, в лямбде.

Колбэк — «вот что сделать, когда будет готово».

Callback hell

```
loadUser(id) { user ->
    loadOrders(user.id) { orders ->
        loadPrices(orders) { prices ->
            calculate(prices) { total ->
                runOnMain {
                    show(user, total)
                }
            }
        }
    }
}
```

Каждый шаг — ещё один уровень вложенности.

Пирамида растёт вправо.

Четыре последовательных шага превратились в пирамиду.

А теперь с ошибками и условиями

```
loadUser(id, ::showError) { user ->
    loadOrders(user.id, ::showError) { orders ->
        if (orders.isEmpty()) {
            show(user, 0)
        } else {
            loadPrices(orders, ::showError) { p ->
                show(user, p.sum())
            }
        }
    }
}
```

onError передаём в каждый вызов.

Обработку ошибок повторяем на каждом шаге.

Цикл по заказам так уже не напишешь.

Ошибки, условия и циклы приходится собирать вручную.

Что теряется с колбэками

Привычное	С колбэками
Код сверху вниз	Пирамида вложенности
try / catch	onError в каждом вызове
for и while	Рекурсия колбэков
Отмена	Флаги и ручные проверки
Понятно, в каком потоке код	Зависит от библиотеки

Колбэки решили проблему потоков, но сломали обычный код.

Future и промисы

```
loadUser(id)
    .thenCompose { user -> loadOrders(user.id) }
    .thenCompose { orders -> loadPrices(orders) }
    .thenApply { prices -> prices.sum() }
    .exceptionally { e -> showError(e); 0 }
```

Future — «результат будет потом».

Пирамиды нет.

Но это уже не if, for и try, а методы библиотеки.

Цепочка лучше пирамиды, но это всё ещё не обычный код.

RxJava

- Всё — поток событий: Observable
- Сотни операторов: map, flatMap, zip, debounce…
- Потоки выполнения задаются через subscribeOn и observeOn
- Долго был стандартом в Android-проектах

Очень мощно, но порог входа высокий, а ошибки трудно искать.

RxJava: загрузить и показать

```
val disposable = api.loadUser(id)
    .flatMap { user -> api.loadOrders(user.id) }
    .map { orders -> orders.sumOf { it.price } }
    .subscribeOn(Schedulers.io())
    .observeOn(AndroidSchedulers.mainThread())
    .subscribe(
        { total -> show(total) },
        { error -> showError(error) },
    )

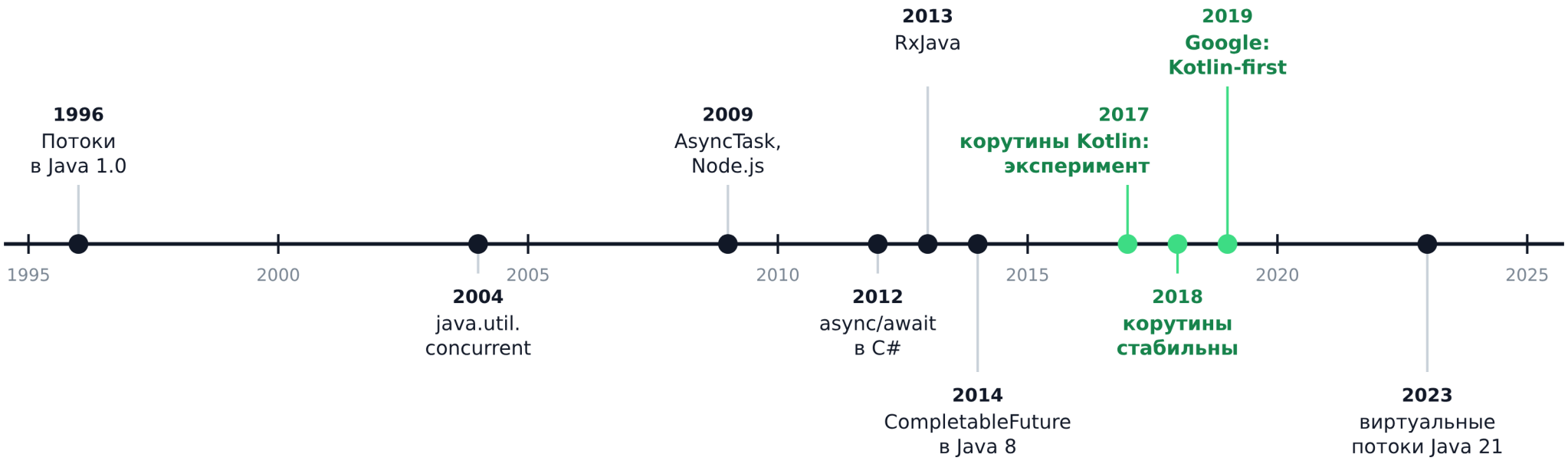
// при закрытии экрана
disposable.dispose()
```

Нужно знать, что такое flatMap, Schedulers, subscribeOn и observeOn.

Забыли dispose — утечка.

Задача та же, но для её решения нужен отдельный язык операторов.

Как к этому шли



Почти тридцать лет поиска: как сделать ожидание дешёвым, а код — простым.

А можно и то, и другое?

Код — как у потока на запрос.
Цена — как у колбэков.

Писать сверху вниз, а пока ждём — не занимать поток. Так бывает?

Корутины: обычный код, который умеет ждать

```
suspend fun loadTotal(id: Int): Int {
    val user = loadUser(id)           // ждём
    val orders = loadOrders(user.id)  // ждём
    val prices = loadPrices(orders)   // ждём
    return prices.sum()
}
```

Выглядит как блокирующий код.

Но пока ждём, поток свободен и делает другое.

Код как у потока на запрос, поведение как у колбэков.

Что такое корутина

- Вычисление, которое может приостановиться и продолжиться позже
- Приостановка не блокирует поток — он берёт другую работу
- Тысячи корутин выполняются на нескольких потоках
- suspend-функцию можно вызвать только из корутины

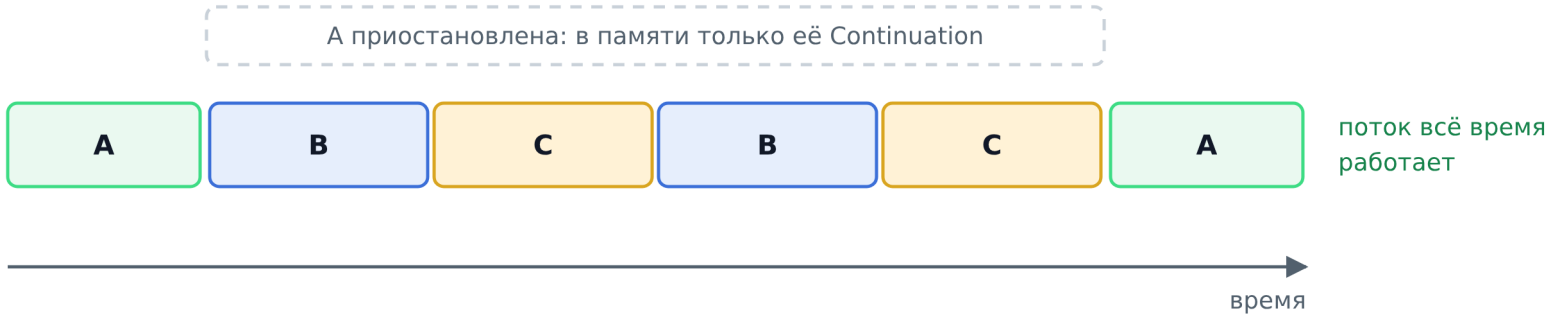
Корутина — не поток. Это работа, которую потоки выполняют по кусочкам.

Блокировать или приостановить

Блокирующее ожидание: Thread.sleep, блокирующий запрос



Приостановка: delay, suspend-функции



Блокировка держит поток. Приостановка его отпускает.

50 000 корутин

```
fun main() = runBlocking {
    repeat(50_000) {
        launch {
            delay(5000)
            print(".")
        }
    }
}
```

50 000 корутин ждут по 5 секунд.

Программа работает около 5 секунд на одном потоке.

С потоками — нехватка памяти или очень долго.

Корутины дешёвые: можно запускать сотни тысяч.

Thread.sleep и delay

	Thread.sleep(1000)	delay(1000)
Поток на время ожидания	Занят	Свободен
Откуда можно вызвать	Откуда угодно	Только из корутины
Можно отменить	Только прерыванием потока	Да, сразу
Цена ожидания	Целый поток	Небольшой объект

Внутри корутины не блокируем поток: delay вместо Thread.sleep.

Как функция останавливается посередине?

И продолжает позже — возможно, на другом потоке?

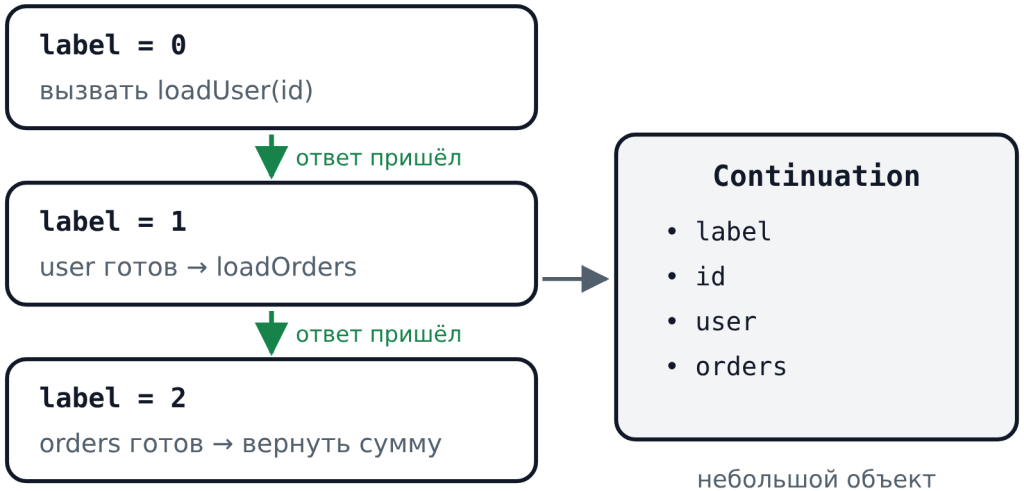
Где хранятся её локальные переменные, пока она «спит»?

Под капотом: конечный автомат

```
suspend fun loadTotal(id: Int): Int {
    val user = loadUser(id)          // 1
    val orders = loadOrders(user.id) // 2
    return orders.sum()
}
```

①② — точки приостановки

Компилятор режет функцию по ним на шаги



небольшой объект
в куче, а не стек

При приостановке функция возвращает COROUTINE_SUSPENDED — поток свободен

Компилятор режет suspend-функцию на шаги и хранит состояние в объекте.

Что делает компилятор

- Добавляет скрытый параметр Continuation
- Делит функцию на шаги по точкам приостановки
- Сохраняет номер шага и переменные в Continuation
- Продолжает с нужного шага, когда результат готов

Корутина — объект в куче, а не поток со стеком в мегабайт.

Где можно вызывать suspend

```
suspend fun loadUser(id: Int): User { ... }

// Обычная функция
fun onClick() {
    loadUser(42)        // ошибка компиляции
}

// Внутри корутины
fun onClick() {
    scope.launch {
        val user = loadUser(42)
        show(user)
    }
}
```

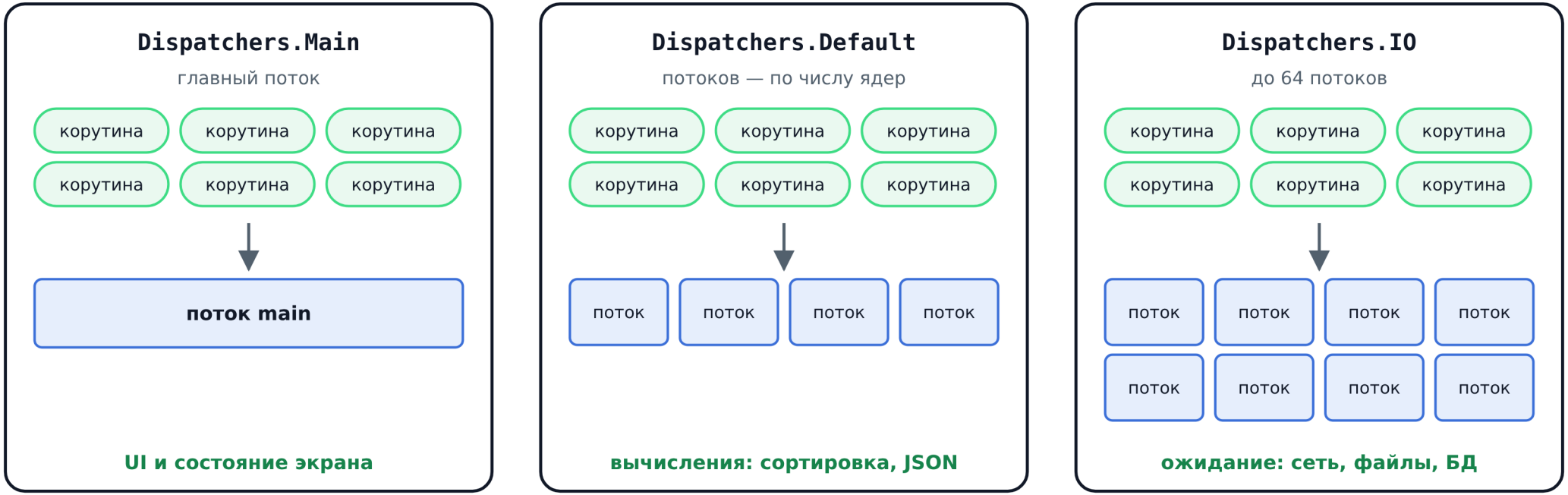
suspend — «может приостановиться».

Приостановиться можно только внутри корутины.

launch — мост из обычного кода.

launch запускает корутину, внутри неё можно вызывать suspend-функции.

Диспетчеры: на каких потоках



Диспетчер решает, на каком потоке выполнится корутина.

Сходить в фон и вернуться

```
fun onLoadClick() {
    viewModelScope.launch {  // Main
        val items = withContext(Dispatchers.IO) {
            api.loadItems()  // IO
        }
        uiState = uiState.copy(items = items)  // Main
    }
}
```

viewModelScope запускает на главном потоке.

withContext — блок на другом диспетчере и возврат назад.

Тот же сценарий, что с потоком и Handler, — без колбэков и ручного возврата.

Параллельно — явно

```
suspend fun loadProfile(id: Int) = coroutineScope {
    val user = async { loadUser(id) }
    val friends = async { loadFriends(id) }
    Profile(user.await(), friends.await())
}
```

Две загрузки идут одновременно.

Время — максимум, а не сумма.

Без async — по очереди.

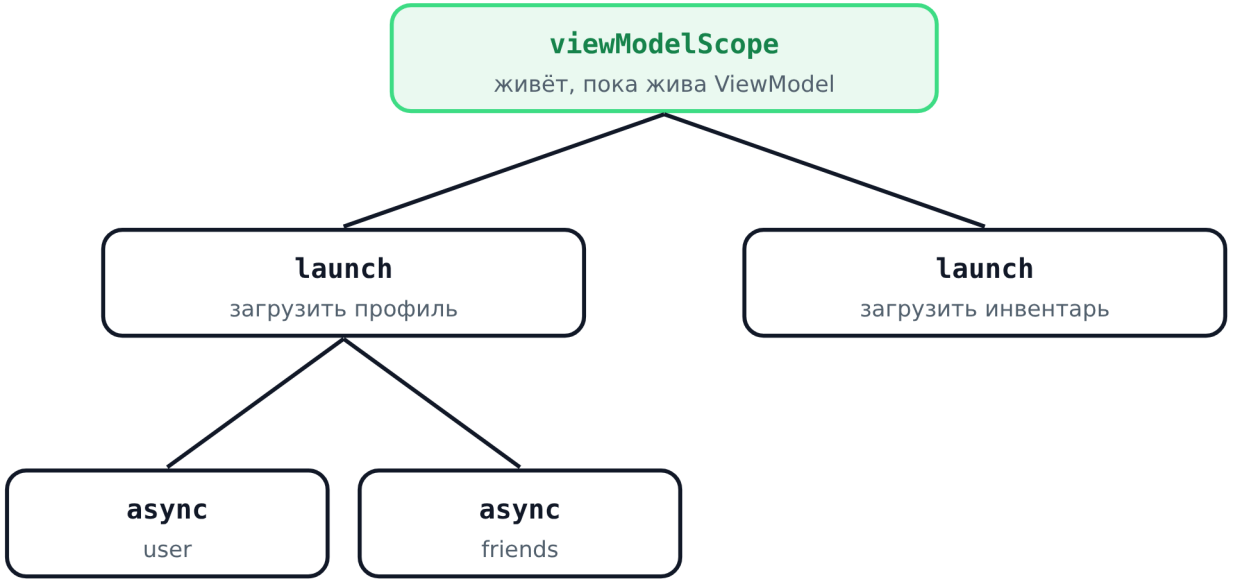
Последовательно — по умолчанию. Конкурентно — только явно, через async.

Пользователь ушёл с экрана

Загрузка ещё идёт.
Кто её остановит?

А если таких загрузок три и одна упала — что с остальными?

Структурная конкурентность



Отмена идёт вниз

onCleared() отменяет scope →
отменяются обе launch →
отменяются все async

Ошибка идёт вверх

ошибка в async(user) →
отменяет соседа friends →
доходит до родителя

Ни одна корутина не «теряется»

У каждой корутины есть родитель. Отмена идёт вниз, ошибка — вверх.

Правила структурной конкурентности

- Корутина запускается в области — scope — с ограниченным временем жизни
- Родитель не завершается, пока не завершены дети
- Отмена родителя отменяет всех детей
- Ошибка ребёнка не теряется — она доходит до родителя

Корутины не «утекают»: у каждой есть владелец.

Что решили корутины

Проблема	Как решено
Поток дорогой	Корутина — небольшой объект
Ожидание держит поток	Приостановка отпускает поток
Пирамида колбэков	Последовательный код
onError в каждом вызове	Обычный try / catch
Отмена вручную	Отмена по scope
Работа «утекает»	Структурная конкурентность
Возврат в главный поток	withContext и Dispatchers.Main

Корутины не добавили новую магию — они сделали ожидание дешёвым, а код — обычным.

Роман Елизаров — автор корутин

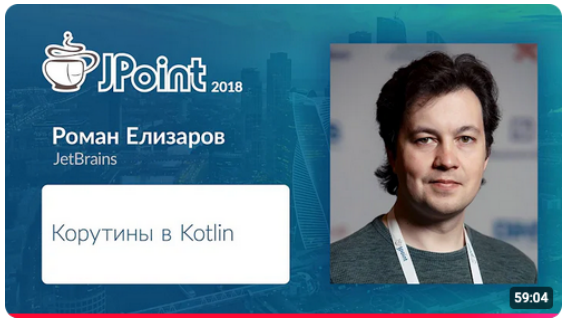


Роман Елизаров

- Выпускник ИТМО, многолетний организатор чемпионатов ICPC
- Около 15 лет в Devexperts: высоконагруженные системы для бирж
- Семь лет в JetBrains: автор корутин и kotlinx.coroutines
- 2020–2023 — руководитель проекта Kotlin
- Преподаёт параллельное программирование в ИТМО

Корутины придумал инженер, который годами писал многопоточные системы.

Корутины от их автора



Корутины в Kotlin
JPoint 2018
на русском, 59 минут
▶ YouTube



Асинхронное программирование с корутинами
JUG.ru, часть 1
на русском, 1 час 6 минут
▶ YouTube



Introduction to Coroutines
KotlinConf 2017
на английском, 46 минут
▶ YouTube



Kotlin Coroutines in Practice
KotlinConf 2018
на английском, 49 минут
▶ YouTube

Нет ничего лучше лекции по корутинам от их создателя.

Решения в дизайне корутин Kotlin

- В языке — только suspend, остальное — в библиотеке
- Последовательно по умолчанию: без await на каждом вызове
- Конкурентность явная: launch и async
- Структурная конкурентность: у каждой корутины есть владелец

Простое — по умолчанию, сложное и опасное — явно.

Та же задача в других языках

Язык	Подход
JavaScript, C#, Python	async / await и промисы
Go	Горутины и каналы
Java 21	Виртуальные потоки
Kotlin	suspend-функции и структурная конкурентность

Идея общая: ожидание должно быть дешёвым. Реализации разные.

Частые заблуждения

Утверждение	На самом деле
«Корутина — это поток»	Работа, которую выполняют потоки
«suspend — значит в фоне»	suspend не меняет поток, это делает диспетчер
«delay и Thread.sleep одинаковы»	sleep держит поток, delay отпускает
«launch — значит параллельно»	На Main все корутины идут по очереди
«GlobalScope.launch — удобно»	Корутина без владельца — утечка

Если сомневаетесь, спросите: на каком потоке это выполняется и кто владелец?

Смотрим вживую

- Thread.sleep(6000) в onClick — замерший экран и ANR
- Гонка счётчика: несколько запусков, разные числа
- 50 000 потоков против 50 000 корутин
- Thread.sleep и delay внутри runBlocking



Каждую демонстрацию сначала предсказываем, потом запускаем.

Практика: от колбэков к корутинам

- Даны suspend-функции загрузки с delay внутри
- Написать loadTotal последовательно и замерить время
- Загрузить независимые части через async и сравнить
- Переписать на корутины готовую пирамиду колбэков

Записать времена и объяснить разницу.

Проверяем понимание

- Чем конкурентность отличается от параллельности?
- Что общего у потоков процесса и что у каждого своё?
- Почему counter++ из двух потоков теряет увеличения?
- Что происходит с потоком, когда корутина вызывает delay?
- Почему suspend-функцию нельзя вызвать из обычной?
- Что будет с корутинами viewModelScope при onCleared()?

Ответ должен объяснять поведение программы, а не только название API.

Что дальше

- viewModelScope и жизненный цикл
- Загрузка данных: состояния загрузки и ошибки
- Flow и StateFlow вместо Compose-состояния в репозитории
- collectAsStateWithLifecycle в Compose

Сегодня — почему корутины такие. Дальше — как ими пользоваться в приложении.

Материалы для повторения

- Kotlin: Coroutines basics, Coroutine context and dispatchers
- Android: Kotlin coroutines on Android
- Роман Елизаров: доклады о корутинах и «Structured concurrency»
- Роб Пайк: «Concurrency is not parallelism»

Ссылки на соответствующие темы есть в заметках к слайдам.